Document Parsing & Chunking

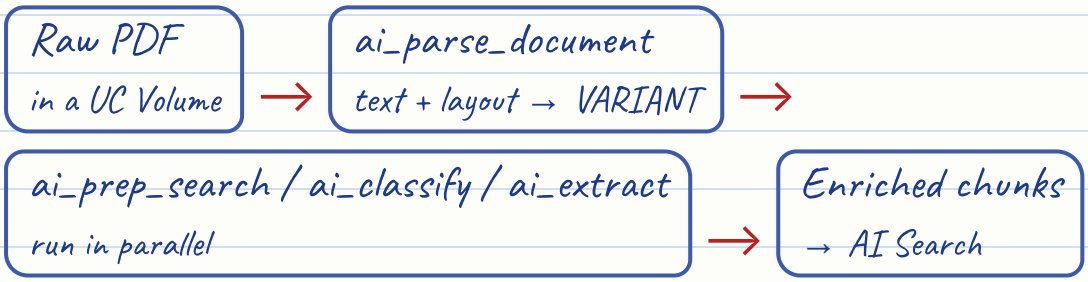
raw PDFs → searchable chunks → AI Search index

★ Naming note: Databricks now calls Vector Search "**AI Search**". Old name = same thing. Expect both on the exam.

Why this matters

- A RAG agent is only as good as what it retrieves.
- So documents must first be **parsed** → **classified/extracted** → **chunked**, then indexed.
- **Shortcut:** Databricks Knowledge Assistant can take UC Volumes / tables directly and does all this parsing + chunking itself. The AI Functions below are for building the pipeline by hand.

The pipeline at a glance



- AI Functions = SQL-callable functions backed by foundation models.
- Labels (classify) + fields (extract) get **joined onto the chunk table** as metadata columns → used for **filtering / ranking** in search.

① ai_parse_document

- Reads PDFs **and images** using AI: OCR, layout analysis, figure descriptions, bounding boxes.
- Returns structured JSON as a VARIANT column.
- Output has `document:elements` (content blocks) and `document:pages` (page by page), plus error status + metadata.
- Pass `'version' → '2.0'` = recommended (better layout, HTML tables; v2 classify/extract examples assume it).

parse every PDF in a volume

```
SELECT path,
       ai_parse_document(content, MAP('version', '2.0')) AS parsed
FROM READ_FILES('/Volumes/cat/schema/docs/', format ⇒ 'binaryFile')
```

variant_explode (un-nesting the parsed JSON)

- Table-valued function: turns a VARIANT **array or object** into rows.
- Output columns: `pos` (position), `key` (field name, NULL for arrays), `value`.
- Object → one row per key. Array → one row per element (key = NULL).
- NULL / non-array input → **no rows**. Want to keep them? use `variant_explode_outer`.
- Use with `LATERAL` (or comma join) to flatten into rows.

one row per parsed element

```
SELECT e.value AS element
FROM parsed_docs t,
     LATERAL variant_explode(t.parsed:document:elements) e
```

② ai_classify vs ai_extract

	ai_classify	ai_extract
Does	picks **one label** from a list you give	pulls out **named fields** you list
Input	text + `["invoice","receipt",…]`	text + `["invoice_id","date","total"]`
Output	single category (read from `response`)	JSON with each field's value
Use for	routing docs / section labels	structured metadata (dates, totals, IDs)

classify + extract, then read the response

```
SELECT content,
  ai_classify(content, '["pricing","amenities","house_rules","other"]'):response[0]
AS category,
  ai_extract(content, '["invoice_id","date","total"]'):response.total            AS
total
FROM elements
```

Tip: passing the full parsed content (not one string) lets the model see more context.

③ ai_prep_search (chunking)

- Splits text into chunks at meaningful (semantic) boundaries using AI, not by fixed character count.
- Per chunk you get: `chunk_to_embed`, `chunk_to_retrieve`, `chunk_id`, `chunk_position`.

simplest form

```
SELECT ai_prep_search(parsed_text) AS chunks FROM parsed_documents
```

Chunking rules of thumb ✎ (exam favourites)

- **Fit the embedding model:** chunks must fit its context window (often ~512 tokens). Extra text is silently truncated.
- **Overlap:** ~10–20% between neighbours, so info at boundaries isn't lost.
- **Granularity trade-off:** small = precise but little context · large = more context but diluted signal.

- **"Lost in the middle"**: LLMs overlook info buried in long contexts → smaller, focused chunks help.

Why two columns? embed vs retrieve

chunk_to_embed	chunk_to_retrieve
chunk + headers / section titles	plain original text
goes to the **embedding model**	goes to the **LLM's context**
answers "what is this chunk about?"	answers "what should the LLM read?"
optimise for findability	optimise for readability

e.g. "The fee is $50" alone is ambiguous; embedding "Cleaning Policy > Late Checkout: The fee is $50" makes it findable. The LLM gets just the clean sentence (no wasted tokens).

4 Chunks → AI Search index

1. read Delta chunk table → 2. embed each chunk (model) → 3. build ANN index (nearest-neighbour) →

4. store synced cols id, text, source_path

First sync can take several minutes (depends on rows + chunk sizes).

Index parameters to remember

Parameter	Means
`endpoint_name`	which search endpoint (compute) serves the index
`source_table_name`	Delta table holding the chunks
`index_name`	full Unity Catalog name for the index
`pipeline_type`	TRIGGERED or CONTINUOUS
`primary_key`	unique row ID → tracks what's been processed
`embedding_source_column`	column that gets embedded (e.g. chunk_to_embed)
`embedding_model_endpoint_name`	embedding model, e.g. `databricks-gte-large-en`
`columns_to_sync`	extra columns stored + returned with results

5 Staying in sync: Change Data Feed

- Turn on CDF for the chunk table: `delta.enableChangeDataFeed = true`
- Required for Delta Sync indexes on standard endpoints.
- CDF logs only the **diff** (insert / update / delete + version + timestamp) → index processes **only changed rows** instead of rebuilding.

enable CDF

```
ALTER TABLE chunks_table
SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
```

Sync modes

Mode	When it syncs	Trade-off
TRIGGERED	only when you call `index.sync()` or click in UI	you control timing, cheaper → batch updates
CONTINUOUS	automatically as the table changes	near real-time, but more compute

One-line recap

- parse → classify/extract → chunk (embed vs retrieve cols) → index → keep fresh with CDF.

Likely exam questions

- Docs change once a day → which sync mode? **TRIGGERED**
- Needed for incremental index sync? **Change Data Feed**
- Chunk longer than model limit? **Silently truncated** → chunk smaller
- Why separate embed/retrieve text? **findability vs readability**
- Label docs into fixed categories? **ai_classify** · pull invoice date/total? **ai_extract**
- No-code path that parses + chunks for you? **Knowledge Assistant**

next lecture → send the screenshots